

GPS L1 IF Carrier NCO Generator — Revision 2

Output Verification & Source Code — Internship Task 2 (Post-Review)

Revision note: This document supersedes the original Task 2 Output_And_Code report. Following supervisor review, two changes were applied to the design: (1) the I_out and Q_out sample width was changed from 12-bit signed to **3-bit signed**, and (2) the previously separate phase_accumulator and sine_lut sub-modules were **merged into a single, self-contained nco_top module** for easier reuse in future tasks. The original 12-bit, multi-file report is now outdated and should be disregarded in favor of this version.

1. Objective

This document records the ModelSim simulation output of the revised, single-file GPS L1 IF Carrier NCO design and lists the complete, final source code, so the results can be reviewed independently of the live simulation environment.

2. Output Verification

2.1 Simulation Transcript

The transcript below shows the self-checking testbench output for the revised design. 12 consecutive samples (3 full carrier cycles) of I (cosine) and Q (sine) are captured and compared against the exact expected repeating pattern for the new 3-bit signed range: 3, 0, -3, 0 for I; 0, 3, 0, -3 for Q.

```
Transcript
# Compile of nco_top.v was successful.
# 2 compiles, 0 failed with no errors.
ModelSim> vsim -gui work.tb_nco_top
# vsim -gui work.tb_nco_top
# Start time: 14:51:51 on Jul 13, 2026
# Loading work.tb_nco_top
# Loading work.nco_top
add wave -position end sim:/tb_nco_top/*
VSIM3> run -all
# =====
# NCO verification (3-bit signed I/Q): 4 samples per cycle
# =====
# sample 0 : I=3  Q=0  (expected I=3 Q=0)
# sample 1 : I=0  Q=3  (expected I=0 Q=3)
# sample 2 : I=-3 Q=0  (expected I=-3 Q=0)
# sample 3 : I=0  Q=-3 (expected I=0 Q=-3)
# sample 4 : I=3  Q=0  (expected I=3 Q=0)
# sample 5 : I=0  Q=3  (expected I=0 Q=3)
# sample 6 : I=-3 Q=0  (expected I=-3 Q=0)
# sample 7 : I=0  Q=-3 (expected I=0 Q=-3)
# sample 8 : I=3  Q=0  (expected I=3 Q=0)
# sample 9 : I=0  Q=3  (expected I=0 Q=3)
# sample 10 : I=-3 Q=0  (expected I=-3 Q=0)
# sample 11 : I=0  Q=-3 (expected I=0 Q=-3)
# =====
# SELF-CHECK PASS: all 12 samples matched exactly.
# =====
# ** Note: $stop      : C:/Users/masha/OneDrive/Desktop/TASK/tb_nco_top.v(86)
# Time: 135 ns  Iteration: 1  Instance: /tb_nco_top
# Break in Module tb_nco_top at C:/Users/masha/OneDrive/Desktop/TASK/tb_nco_top.v line 86
```

Figure 1: Simulation transcript (3-bit signed I/Q), self-check PASS.

2.2 Waveform (ModelSim)

The waveform below shows clk, reset, I_out and Q_out, with each output's individual bits [2:0] also expanded for inspection. The same 4-clock quadrature staircase pattern is visible as before, now scaled to the 3-bit signed range (+3, 0, -3, 0) instead of the previous 12-bit range (+2047, 0, -2047, 0).

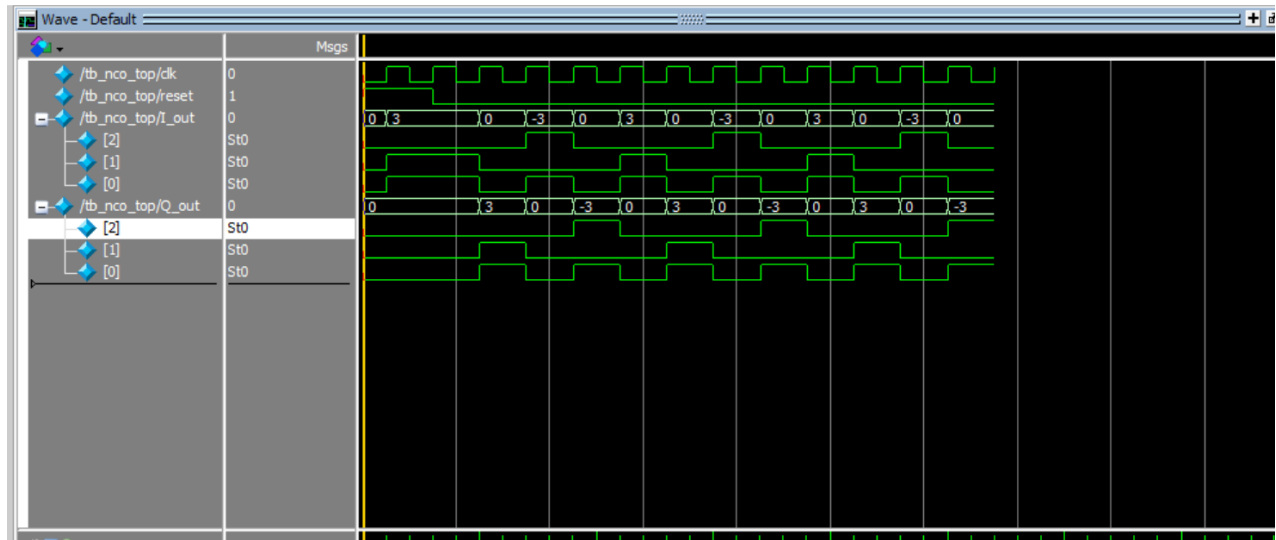


Figure 2: ModelSim waveform, 3-bit signed I_out/Q_out.

2.3 Result Comparison

Sample	I_out (simulated)	Q_out (simulated)	Expected I / Q	Match
0	3	0	3 / 0	PASS
1	0	3	0 / 3	PASS
2	-3	0	-3 / 0	PASS
3	0	-3	0 / -3	PASS
4	3	0	3 / 0	PASS
5	0	3	0 / 3	PASS
6	-3	0	-3 / 0	PASS
7	0	-3	0 / -3	PASS
8	3	0	3 / 0	PASS
9	0	3	0 / 3	PASS
10	-3	0	-3 / 0	PASS
11	0	-3	0 / -3	PASS

SELF-CHECK PASS: all 12 samples matched exactly, with 0 mismatches reported by the testbench.

3. Source Code

The complete, final source code is listed below. Note that only 2 files remain (previously 4), since the phase accumulator and sine lookup table logic are now merged directly into nco_top.v.

nco_top.v

Purpose: Single, self-contained NCO module -- phase accumulator, LUT addressing (direct + 90-degree offset), and the sine lookup function are all merged into one file, producing 3-bit signed I/Q outputs.

```
// =====
// Module: nco_top
// Description: GPS L1 IF Carrier NCO -- single, self-contained module
//              (phase accumulator + sine lookup merged in, per
//              supervisor's request, for easier reuse in future tasks)
//
// Fclk = 16.368 MHz, Fout = 4.092 MHz => Fclk/Fout = exactly 4
// I_out (cosine) and Q_out (sine) are now 3-bit signed, per
// supervisor's requested change (previously 12-bit signed).
// =====
module nco_top #(
    parameter ACC_WIDTH      = 32,    // phase accumulator width
    parameter LUT_ADDR_WIDTH = 6,     // 64-entry sine table
    parameter OUT_WIDTH      = 3      // signed sample width (was 12, now 3)
)(
    input  clk,
    input  reset,
    input  [ACC_WIDTH-1:0] phase_step,

    output signed [OUT_WIDTH-1:0] I_out,    // cosine (in-phase)
    output signed [OUT_WIDTH-1:0] Q_out    // sine   (quadrature)
);

// ---- phase accumulator (was phase_accumulator.v) ----
reg [ACC_WIDTH-1:0] phase;

always @(posedge clk) begin
    if (reset)
        phase <= {ACC_WIDTH{1'b0}};
    else
        phase <= phase + phase_step;    // wraps automatically (modulo 2^ACC_WIDTH)
end

// ---- LUT addressing: top bits = angle, +quarter-table = +90 deg ----
wire [LUT_ADDR_WIDTH-1:0] addr_q = phase[ACC_WIDTH-1 -: LUT_ADDR_WIDTH];
wire [LUT_ADDR_WIDTH-1:0] addr_i = addr_q + (1 << (LUT_ADDR_WIDTH-2));

// ---- sine lookup (was sine_lut.v) ----
// 64-entry sine table, one full cycle (0 to 360 degrees).
// Values scaled to +/-3 (3-bit signed, symmetric full scale).
// Generated offline from: round(3 * sin(2*pi*addr/64))
function signed [OUT_WIDTH-1:0] sine_value;
    input [LUT_ADDR_WIDTH-1:0] addr;
    begin
        case(addr)
            6'd0  : sine_value = 3'sd0;
            6'd1  : sine_value = 3'sd0;
            6'd2  : sine_value = 3'sd1;
            6'd3  : sine_value = 3'sd1;
            6'd4  : sine_value = 3'sd1;
            6'd5  : sine_value = 3'sd1;
            6'd6  : sine_value = 3'sd2;
            6'd7  : sine_value = 3'sd2;
            6'd8  : sine_value = 3'sd2;
            6'd9  : sine_value = 3'sd2;
            6'd10 : sine_value = 3'sd2;
```

```

        6'd11 : sine_value = 3'sd3;
        6'd12 : sine_value = 3'sd3;
        6'd13 : sine_value = 3'sd3;
        6'd14 : sine_value = 3'sd3;
        6'd15 : sine_value = 3'sd3;
        6'd16 : sine_value = 3'sd3;
        6'd17 : sine_value = 3'sd3;
        6'd18 : sine_value = 3'sd3;
        6'd19 : sine_value = 3'sd3;
        6'd20 : sine_value = 3'sd3;
        6'd21 : sine_value = 3'sd3;
        6'd22 : sine_value = 3'sd2;
        6'd23 : sine_value = 3'sd2;
        6'd24 : sine_value = 3'sd2;
        6'd25 : sine_value = 3'sd2;
        6'd26 : sine_value = 3'sd2;
        6'd27 : sine_value = 3'sd1;
        6'd28 : sine_value = 3'sd1;
        6'd29 : sine_value = 3'sd1;
        6'd30 : sine_value = 3'sd1;
        6'd31 : sine_value = 3'sd0;
        6'd32 : sine_value = 3'sd0;
        6'd33 : sine_value = 3'sd0;
        6'd34 : sine_value = -3'sd1;
        6'd35 : sine_value = -3'sd1;
        6'd36 : sine_value = -3'sd1;
        6'd37 : sine_value = -3'sd1;
        6'd38 : sine_value = -3'sd2;
        6'd39 : sine_value = -3'sd2;
        6'd40 : sine_value = -3'sd2;
        6'd41 : sine_value = -3'sd2;
        6'd42 : sine_value = -3'sd2;
        6'd43 : sine_value = -3'sd3;
        6'd44 : sine_value = -3'sd3;
        6'd45 : sine_value = -3'sd3;
        6'd46 : sine_value = -3'sd3;
        6'd47 : sine_value = -3'sd3;
        6'd48 : sine_value = -3'sd3;
        6'd49 : sine_value = -3'sd3;
        6'd50 : sine_value = -3'sd3;
        6'd51 : sine_value = -3'sd3;
        6'd52 : sine_value = -3'sd3;
        6'd53 : sine_value = -3'sd3;
        6'd54 : sine_value = -3'sd2;
        6'd55 : sine_value = -3'sd2;
        6'd56 : sine_value = -3'sd2;
        6'd57 : sine_value = -3'sd2;
        6'd58 : sine_value = -3'sd2;
        6'd59 : sine_value = -3'sd1;
        6'd60 : sine_value = -3'sd1;
        6'd61 : sine_value = -3'sd1;
        6'd62 : sine_value = -3'sd1;
        6'd63 : sine_value = 3'sd0;
        default: sine_value = 3'sd0;
    endcase
end
endfunction

assign Q_out = sine_value(addr_q);    // sine
assign I_out = sine_value(addr_i);    // cosine (quarter-table ahead)

endmodule

```

tb_nco_top.v

Purpose: Self-checking testbench; drives the merged design with the phase_step for 4.092 MHz and compares 12 captured I/Q samples against the exact expected 3-bit signed repeating pattern.

```

`timescale 1ns/1ps
// =====
// Testbench: nco_top (merged, 3-bit signed I/Q version)
//
// Fclk = 16.368 MHz, Fout = 4.092 MHz => Fclk/Fout = exactly 4
// I/Q are now 3-bit signed (range -4..+3), full scale = 3.
//   I (cosine): 3,  0, -3,  0,  3,  0, -3,  0, ...
//   Q (sine)  : 0,  3,  0, -3,  0,  3,  0, -3, ...
// =====
module tb_nco_top;

    localparam ACC_WIDTH      = 32;
    localparam LUT_ADDR_WIDTH = 6;
    localparam OUT_WIDTH      = 3;
    localparam FS              = 3;    // full scale for 3-bit signed

    // phase_step = 2^ACC_WIDTH * (Fout/Fclk) = 2^32 * (1/4) = 2^30
    localparam [ACC_WIDTH-1:0] PHASE_STEP = 32'd1073741824;

    reg clk;
    reg reset;
    wire signed [OUT_WIDTH-1:0] I_out;
    wire signed [OUT_WIDTH-1:0] Q_out;

    integer i;
    integer errors;

    reg signed [OUT_WIDTH-1:0] expected_I [0:3];
    reg signed [OUT_WIDTH-1:0] expected_Q [0:3];

    nco_top #(
        .ACC_WIDTH(ACC_WIDTH),
        .LUT_ADDR_WIDTH(LUT_ADDR_WIDTH),
        .OUT_WIDTH(OUT_WIDTH)
    ) DUT (
        .clk(clk),
        .reset(reset),
        .phase_step(PHASE_STEP),
        .I_out(I_out),
        .Q_out(Q_out)
    );

    always #5 clk = ~clk;

    initial begin
        expected_I[0] = FS; expected_Q[0] = 0;
        expected_I[1] = 0; expected_Q[1] = FS;
        expected_I[2] = -FS; expected_Q[2] = 0;
        expected_I[3] = 0; expected_Q[3] = -FS;
    end

    initial begin
        clk      = 0;
        reset    = 1;
        errors    = 0;

        @(posedge clk);
        @(posedge clk);
        reset = 0;
        #1;

        $display("=====");
        $display(" NCO verification (3-bit signed I/Q): 4 samples per cycle");
        $display("=====");

        for (i = 0; i < 12; i = i + 1) begin
            #1;
            $display("sample %0d : I=%0d Q=%0d (expected I=%0d Q=%0d)",
                i, I_out, Q_out, expected_I[i%4], expected_Q[i%4]);
        end
    end
endmodule

```

```

        if (I_out != expected_I[i%4] || Q_out != expected_Q[i%4]) begin
            $display("    ^^ MISMATCH at sample %0d", i);
            errors = errors + 1;
        end

        @(posedge clk);
    end

    $display("=====");
    if (errors == 0)
        $display(" SELF-CHECK PASS: all 12 samples matched exactly.");
    else
        $display(" SELF-CHECK FAIL: %0d mismatches found.", errors);
    $display("=====");

    $stop;
end

endmodule

```